SYSTEM DESIGN GUIDE · JOE'S ROUND

Five Designs, *One Interview*

Everything you need to walk into Joe's system design + pair programming round. Each topic covers architecture, schema, tradeoffs, questions Joe will ask, and questions you should ask.

01
AI Collections Agent
State machines · LLM orchestration

02
Multi-Tenant Rules Engine
Config versioning · Runtime evaluation

03
ERP Integration Layer
Webhooks vs polling · Idempotent sync

04
Audit Log System
Append-only · Event sourcing

05
Remittance Matching
Fuzzy matching · Cash application

01

MOST LIKELY TO COME UP FIRST

AI Collections Agent Architecture

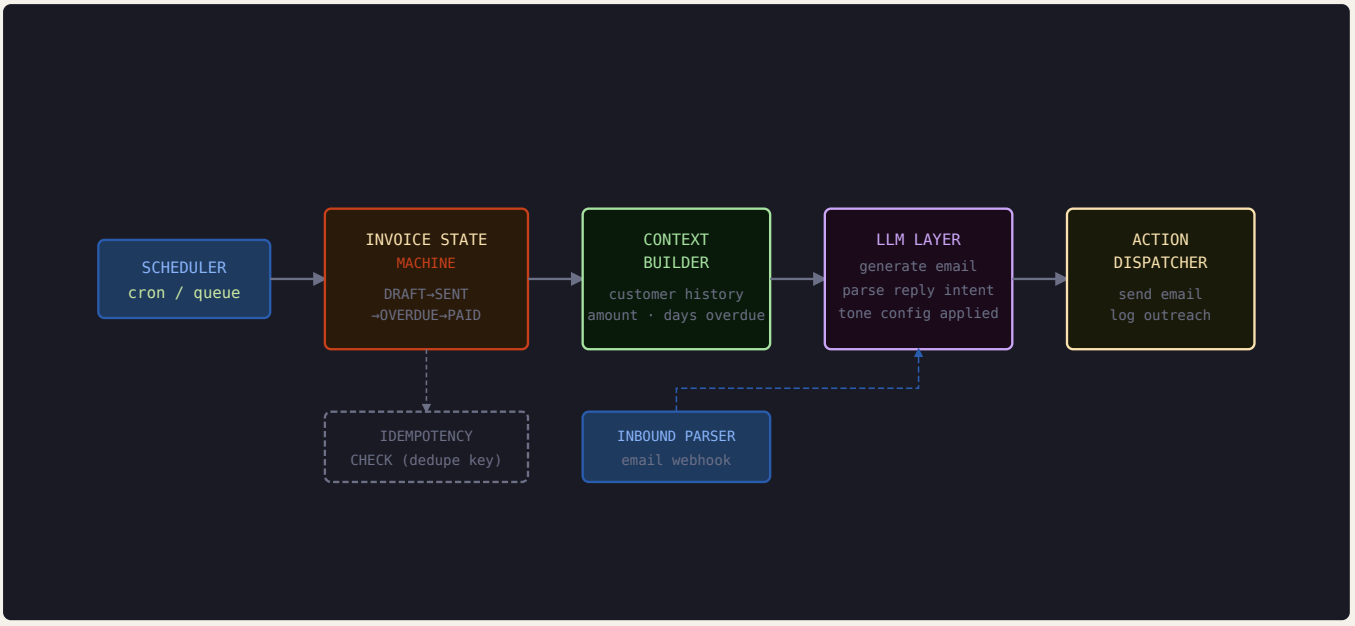
Core differentiator | State Machine | LLM Orchestration | Idempotency | Scheduling | Event-Driven

differentiator — a wrong action (double-emailing, dunning a customer who just paid) destroys trust and is a core design challenge.

THE CORE PROBLEM

How do you build an agent that decides *when* to reach out, *what* to say, *how* to interpret a reply, and *what action* to take next — all while guaranteeing consistency with the true payment state?

ARCHITECTURE OVERVIEW



INVOICE STATE MACHINE

DRAFT → SENT → OVERDUE → FOLLOW-UP 1 → FOLLOW-UP 2 → ESCALATED → PAID

Each state transition is recorded as an immutable event. The agent's scheduler reads current state and determines if a follow-up is due based on tenant configuration (e.g., "follow up 3 days after due date, then every 5 days").

KEY SCHEMA

```
-- Invoice with state tracking TABLE invoices ( id UUID PRIMARY KEY, tenant_id UUID
NOT NULL, customer_id UUID NOT NULL, amount_cents BIGINT NOT NULL, status
```

```
idempotency_key TEXT UNIQUE -- prevent double sends ); -- outreach log (append-
only) TABLE outreach_events ( id UUID PRIMARY KEY, invoice_id UUID REFERENCES
invoices, channel ENUM('email','sms','portal'), direction
ENUM('outbound','inbound'), content TEXT, llm_model TEXT, intent TEXT, -- parsed:
'promise_to_pay' | 'dispute' | 'ignore' sent_at TIMESTAMPTZ );
```

CRITICAL DESIGN CONSIDERATIONS

RACE CONDITION PREVENTION

Payment ↔ Outreach Race

Check payment status immediately before dispatching outreach. Use a DB-level lock or optimistic version field — never send a dunning email to someone who paid 2 minutes ago.

IDEMPOTENCY

Dedupe Key on Every Send

Compose key as `invoice_id + followup_count + date`. If the job retries after a crash, the same key prevents a second email from going out.

LLM RELIABILITY

Fallback to Templates

LLM calls can fail or timeout. Keep a template fallback per follow-up number so the email always goes out, even if personalization fails.

HUMAN ESCALATION

Escalation Queue

After N follow-ups with no response, or if the LLM detects a dispute, move to a human review queue. The agent should never autonomously handle disputes.

TRADEOFFS

DECISION	OPTION A	OPTION B	MONK'S LIKELY CHOICE
Scheduling	**Cron job polling DB** — simple, no infra	Message queue (BullMQ / SQS) — scalable	Cron at early stage → queue as they scale
Reply parsing	**LLM classification** — flexible, handles natural language	Regex/rules — fast, predictable but brittle	LLM with regex fallback for obvious cases
State storage	**Status column on invoices** — simple queries	Event sourcing — full history but complex	Status column + outreach_events log

Outreach timing	**next_followup_at cursor** — one row per invoice	Scheduled jobs per invoice — noisy at scale	Cursor approach, batch poll every N minutes

🎙 QUESTIONS JOE WILL ASK YOU

- → How do you prevent double-sending a follow-up email?
- → A customer marks an invoice disputed mid-outreach — what happens?
- → How does the system know when to stop following up?
- → How would you test the agent logic without actually sending emails?
- → If the LLM is down, does outreach stop entirely?
- → How do you handle timezone-aware scheduling?

🙋 QUESTIONS YOU SHOULD ASK JOE

- → How does Monk currently handle disputed invoices — is there a human loop?
- → What's the current outreach channel mix — email only, or SMS too?
- → Do tenants configure their own follow-up cadence or is it fixed?
- → What happens to agent state when a customer partially pays?
- → How do you evaluate whether an LLM-written email is "good"?
- → Are there compliance constraints on how many times you can contact someone?

⚡ CRITICAL KNOWLEDGE TO HAVE COLD

- ✦ Idempotency keys & why they matter
- ✦ State machine pattern & valid transitions
- ✦ Optimistic vs pessimistic locking
- ✦ LLM prompt design for classification
- ✦ Outbox pattern for reliable event dispatch
- ✦ Dead letter queues for failed jobs
- ✦ SMTP webhook for inbound email parsing
- ✦ Retry logic with exponential backoff

💡 FOUNDER INSIGHT — Say This

> *here, and I'd design the payment check as a pre-flight gate before every single outreach dispatch."*

02

MULTI-TENANT ARCHITECTURE

Multi-Tenant Rules Engine & Configuration Layer

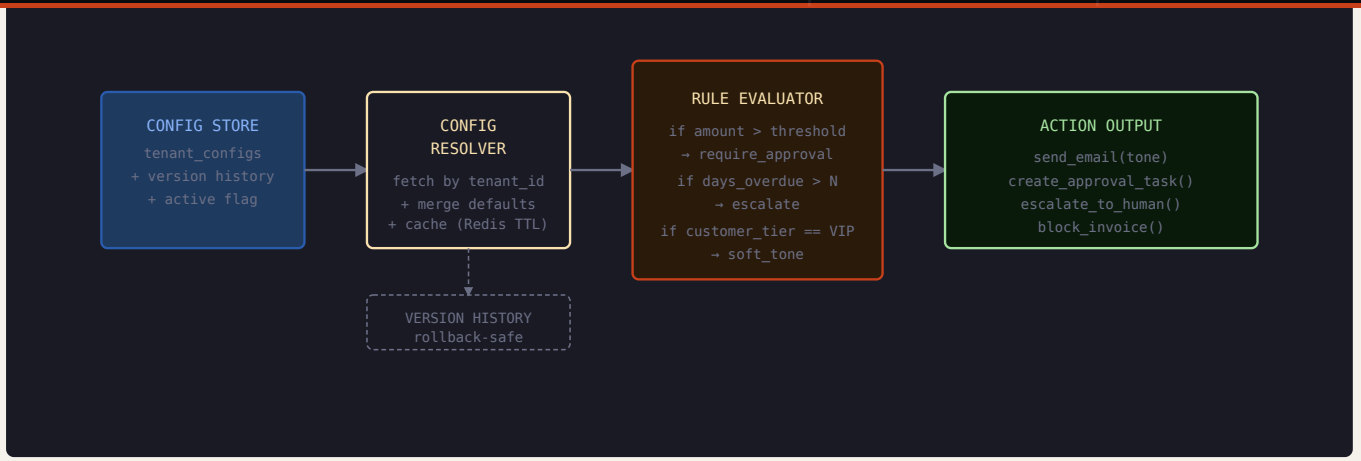
Startup scaling challenge | Multi-tenancy | Config Versioning | Runtime Evaluation | No-Code DSL

Every Monk customer has different needs: different invoice tone, different escalation thresholds, different approval workflows, different exception handling. The rules engine lets tenants configure behavior without code deploys — and lets Monk evolve configurations safely without breaking running workflows.

THE PROBLEM IN ONE SENTENCE

How do you let each of N customers define their own business rules, evaluate them at runtime, version them safely, and test changes without affecting live AR workflows?

ARCHITECTURE



SCHEMA DESIGN

```
-- Tenant configuration (versioned) TABLE tenant_configs ( id UUID PRIMARY KEY,
tenant_id UUID NOT NULL, version INT NOT NULL, is_active BOOLEAN DEFAULT false,
config JSONB NOT NULL, -- all rules serialized created_at TIMESTAMPTZ, created_by
UUID, UNIQUE(tenant_id, version) ); -- Example JSONB config structure { "tone":
"professional", "followup_cadence_days": [3, 7, 14], "escalation_threshold_days":
30, "approval_required_above_cents": 1000000, "vip_customers": ["cust_abc",
"cust_xyz"], "exceptions": { "holiday_pause": { "from": "12-20", "to": "01-02" } }
}
```

VERSIONING STRATEGY — THE CRITICAL PART

PROBLEM

In-flight Workflows

If you update a config mid-workflow, invoices in FOLLOW-UP 2 state might suddenly skip to FOLLOW-UP 3 or change tone mid-sequence. You need each workflow instance to "lock" its config version at start.

SOLUTION

Config Snapshot on Workflow Start

When a collections sequence begins, snapshot the active config version onto the invoice record. Subsequent agent decisions for that invoice use the snapshotted version, not the current active one.

CACHING

Redis TTL Cache

Cache active config per tenant with a short TTL (30–60s). When a new version is activated,

TESTING

Shadow Mode / Dry Run

New config versions can be tested in shadow mode — run the evaluator against live invoices,

TRADEOFFS

DECISION	OPTION A	OPTION B	RECOMMENDATION
Rule storage	**JSONB in Postgres** — easy to query, schema-free	Separate rules tables — typed, relational	JSONB early-stage, typed tables as rules grow complex
Rule evaluation	**Code-side evaluator** — full control, testable	External rules engine (Drools, OPA) — overkill early	Code evaluator with a clean interface to swap later
Config activation	Immediate swap — simple but risks in-flight issues	**Versioned + snapshot** — safe but more complex	Versioned; in-flight workflows pin to their version
Tenant defaults	Hardcode in app code — fast but inflexible	**Default config record per tenant** — override-able	Default record + per-tenant override, merge at resolve time

🎙 QUESTIONS JOE WILL ASK YOU

- → What happens if a tenant updates their config mid-sequence?
- → How would you validate a config before it goes live?
- → How do you handle config rules that conflict with each other?
- → How does a new tenant get their first config?
- → What if a customer wants a rule that no other customer uses?

🙋 QUESTIONS YOU SHOULD ASK JOE

- → How do customers currently configure Monk — UI, Slack, onboarding call?
- → Are there config changes that should take effect immediately vs next billing cycle?
- → Do you have a concept of "global defaults" that apply unless overridden?
- → What's the most complex rule a customer has ever requested?

⚡ CRITICAL KNOWLEDGE TO HAVE COLD

- Feature flags pattern (same concept)
- Strategy pattern for rule evaluation
- Cache invalidation strategies
- Config schema validation (Zod / JSON Schema)
- Soft delete vs hard delete for config versions
- Tenant isolation patterns in shared DB

💡 **FOUNDER INSIGHT — Say This**

"Early stage I'd model this as a JSONB blob — it's flexible and you can evolve it fast. The important thing isn't the storage format, it's pinning in-flight workflows to a config snapshot so a tenant changing their rules on a Tuesday afternoon doesn't scramble outstanding collections."

03

THIRD-PARTY RELIABILITY

ERP / Third-Party Integration Layer

Reliability challenge | Webhooks | Idempotent Sync | Eventual Consistency | Stripe | QuickBooks | Coupa/Ariba

Monk integrates with QuickBooks, Stripe, NetSuite, Coupa, and Ariba. Each has a different data model, rate limits, and reliability profile. This integration layer is responsible for keeping Monk's view of invoices, payments, and customers in sync — without data loss, duplication, or corruption.

CORE CHALLENGES

CHALLENGE 1

Webhooks vs Polling

CHALLENGE 2

Schema Differences

unreliable. You need a strategy for each and a fallback reconciliation job.

canonical internal model and adapters per integration.

CHALLENGE 3

Eventual Consistency

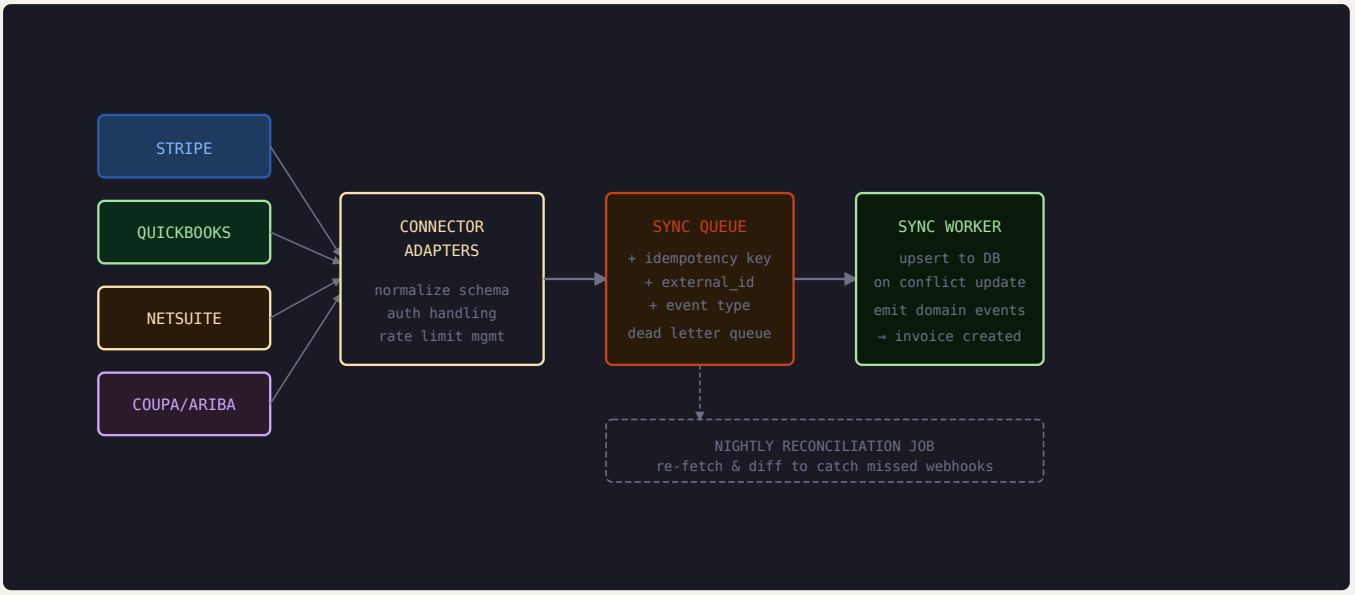
A payment confirmed in Stripe may take seconds to minutes to appear in Monk's DB. A follow-up agent must not fire during this window.

CHALLENGE 4

Failure Recovery

The integration must survive: network failures, API rate limits, schema changes in the third party, and auth token expiry — without losing data.

ARCHITECTURE



CANONICAL INTERNAL MODEL

```
-- Canonical invoice (source-agnostic) TABLE invoices ( id UUID PRIMARY KEY,
tenant_id UUID, external_id TEXT, -- ID in source system source
ENUM('stripe','quickbooks','netsuite','manual'), external_synced_at TIMESTAMPTZ,
amount_cents BIGINT, status TEXT, raw_payload JSONB -- original source payload, for
debugging ); -- Integration credentials (per tenant, per source) TABLE integrations
( id UUID, tenant_id UUID, provider TEXT, access_token TEXT -- encrypted at rest,
refresh_token TEXT, token_expires_at TIMESTAMPTZ, last_sync_at TIMESTAMPTZ );
```

WEBHOOK VS POLLING DECISION TREE

Stripe	**Webhooks (primary)**	Reliable, low latency, event-rich	Polling /invoices?created_after=X
QuickBooks	Webhooks + polling	Webhooks exist but can miss events	Nightly full reconciliation
NetSuite	**Polling (primary)**	No native webhooks for many record types	Custom saved searches via API
Coupa/Ariba	Portal scraping + API	Procurement portals, often no API	Human-in-loop for edge cases

IDEMPOTENT UPSERT PATTERN

```
-- Never INSERT blindly — always upsert by external_id + source INSERT INTO
invoices (id, tenant_id, external_id, source, amount_cents, status) VALUES ($1, $2,
$3, $4, $5, $6) ON CONFLICT (tenant_id, external_id, source) DO UPDATE SET
amount_cents = EXCLUDED.amount_cents, status = EXCLUDED.status, external_synced_at
= NOW() WHERE invoices.external_synced_at < EXCLUDED.external_synced_at; -- The
WHERE clause prevents stale data overwriting newer data
```

🎙 QUESTIONS JOE WILL ASK YOU

- → A webhook fires twice for the same payment — what happens?
- → QuickBooks is down for 2 hours. How do you recover?
- → How do you handle a Stripe invoice that's been partially refunded?
- → How do you detect when a webhook missed an event?
- → OAuth tokens expire — how does your system re-authenticate?
- → How do you test integrations without real third-party credentials?

🙋 QUESTIONS YOU SHOULD ASK JOE

- → What's the most painful integration you've built so far — Coupa? Ariba?
- → Do customers typically connect one ERP or multiple sources?
- → When there's a conflict between Monk's view and the ERP's view, which wins?
- → How do you currently handle OAuth refresh token rotation?

- Webhook signature verification (HMAC)
- Upsert with ON CONFLICT in Postgres
- OAuth2 flow & token refresh pattern
- Exponential backoff for failed API calls
- Dead letter queues & retry semantics
- At-least-once vs exactly-once delivery
- Canonical data model / adapter pattern
- Distributed locking for sync jobs

💡 **FOUNDER INSIGHT — Say This**

"The thing I'd insist on from day one is storing the raw payload alongside the normalized record. When a customer says 'Monk missed a payment,' you need to be able to prove exactly what the ERP sent you, at what timestamp, and what you did with it. The raw_payload JSONB column is your debugging lifeline."

04

JUST SHIPPED FEB 2026 — VERY LIKELY TOPIC

Audit Log System

Recently shipped by Monk | Append-Only | Event Sourcing | Compliance | Tamper-Proof | JSONB

Monk just shipped their Audit Log feature in February 2026. This means it's either a topic Joe wants to explore architecturally, or — more likely for the pair programming segment — they ask you to implement the core of it. Every financial action (invoice sent, payment matched, agent outreach, config change) needs to be logged in an immutable, queryable, trustworthy record.

FUNCTIONAL

What Gets Logged

Every state-changing action: invoice create/update/send, payment match, agent email sent/received, config change, user login, ERP sync. Basically every write operation.

FUNCTIONAL

Queryability

Must support: "show me all actions on invoice X," "show me everything user Y did this month," "show me all config changes for tenant Z." Filtering by actor, resource, time range.

NON-FUNCTIONAL

Immutability

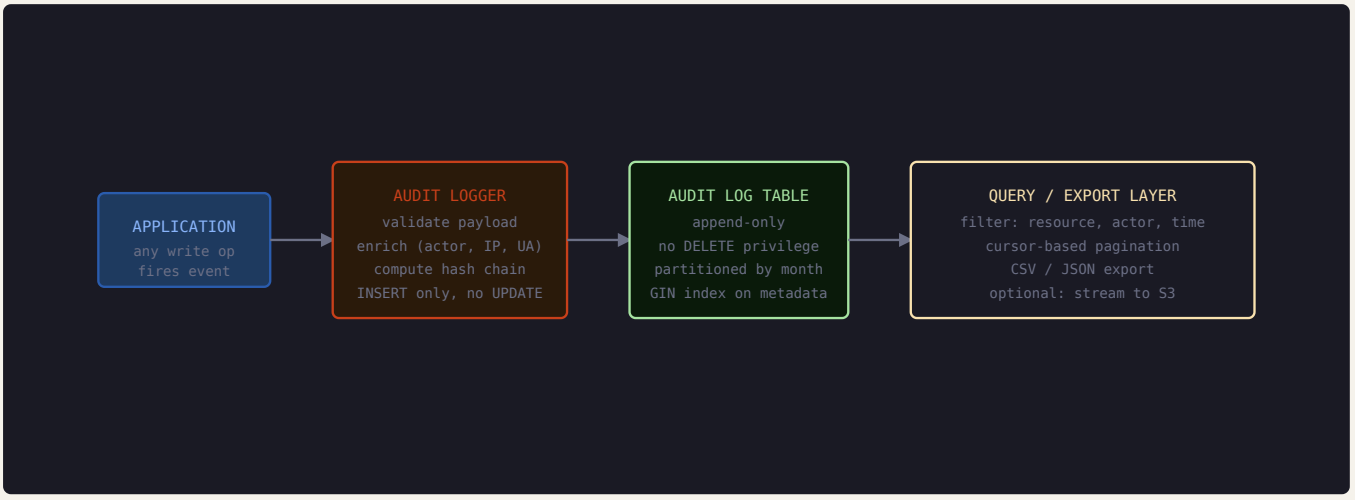
Audit records must never be updated or deleted — even by admins. Any correction is a new record. This is both a technical requirement and a trust requirement for finance teams.

NON-FUNCTIONAL

Tamper Detection

For compliance-sensitive customers, you might hash each record with the previous record's hash — a Merkle chain — so any tampering is detectable. (Mention this as an advanced option.)

ARCHITECTURE



SCHEMA — THE CORE DESIGN

```
-- Generic audit log (EAV-style for flexibility) TABLE audit_logs ( id UUID PRIMARY
KEY DEFAULT gen_random_uuid(), tenant_id UUID NOT NULL, -- WHO actor_type
ENUM('user','system','agent','api_key') NOT NULL, actor_id UUID, actor_email TEXT,
-- WHAT action TEXT NOT NULL, -- 'invoice.sent' | 'payment.matched' |
'config.updated' resource_type TEXT NOT NULL, -- 'invoice' | 'payment' |
'tenant_config' resource_id UUID, -- PAYLOAD metadata JSONB, -- before/after
snapshots, relevant IDs -- WHEN & WHERE created_at TIMESTAMPTZ NOT NULL DEFAULT
```

```
PARTITION BY RANGE (created_at); -- Enforce append-only at DB level REVOKE UPDATE,
DELETE ON audit_logs FROM app_user; -- Fast queries: filter by tenant + resource
CREATE INDEX idx_audit_tenant_resource ON audit_logs (tenant_id, resource_type,
resource_id, created_at DESC); CREATE INDEX idx_audit_actor ON audit_logs
(tenant_id, actor_id, created_at DESC);
```

ACTION NAMING CONVENTION

Use a consistent `resource.verb` pattern for the action field. This makes queries, dashboards, and alerting trivially easy.

```
'invoice.created' 'invoice.sent' 'invoice.voided' 'payment.matched'
'payment.unmatched' 'payment.refunded' 'outreach.sent' 'outreach.replied'
'outreach.bounced' 'config.updated' 'config.activated' 'config.rolledback'
'user.login' 'user.invited' 'integration.connected'
```

TRADEOFFS

DECISION	OPTION A	OPTION B	RECOMMENDATION
Separate table vs same DB	**Same DB** — simpler, joins possible	Separate append-only store — isolation	Same DB early; move to separate if volume demands
Write path	**Synchronous in same transaction** — consistent	Async queue — lower latency but can lose events	Synchronous; audit must not be lossy
Partitioning	**Monthly range partitions** — prune old data easily	No partitioning — simpler early on	Partition from day one; retrofit is painful
Hash chain	No hash — simpler, still append-only	**Merkle chain** — tamper-proof, compliance story	Add hash chain if enterprise/SOC2 customers require it

🎙 QUESTIONS JOE WILL ASK YOU

🙋 QUESTIONS YOU SHOULD ASK JOE

- → An audit log grows unboundedly — how do you manage that?
- → How do you query "show me all changes to invoice X in chronological order"?
- → The audit log write is in the same transaction as the action — what if it fails?
- → How would you surface audit logs in the UI — what would the table look like?
- → How do you handle PII in audit logs (customer email in metadata)?

- → Do you have compliance requirements (SOC2, GDPR) that shape design?
- → How long should audit records be retained?
- → Should customers be able to export their full audit history?

⚡ CRITICAL KNOWLEDGE TO HAVE COLD

- ✦ REVOKE DELETE/UPDATE in Postgres
- ✦ Range partitioning by timestamp
- ✦ GIN index on JSONB fields
- ✦ Cursor-based pagination (not OFFSET)
- ✦ GDPR right-to-erasure vs audit immutability tension
- ✦ Merkle hash chains (basic concept)
- ✦ Outbox pattern for reliable log writes
- ✦ Before/after state diffing

💡 **FOUNDER INSIGHT — Say This**

"Audit logs aren't just a compliance checkbox — for a finance product they're a debugging superpower. When a customer says 'who changed my escalation threshold last Tuesday,' you answer it in 3 seconds. I'd make the audit log customer-facing early so it becomes a trust-building feature, not just an internal tool."

05

THEIR BLOG'S HEADLINE FEATURE

Remittance Matching / Cash Application

Blog headline: "One-day cash application" | Fuzzy Matching | Confidence Scoring | Human Review Queue | AI Extraction

When a payment arrives, Monk needs to figure out which invoice(s) it belongs to. This sounds simple — but in practice, customers pay with wrong amounts, vague bank memo lines ("INV PAYMENT"), combine multiple invoices into one wire, or send partial payments. This is the "cash application" problem, and it's where 39% of AR slowdown comes from according to Monk's own data.

THE MATCHING PROBLEM DECOMPOSED

HARD CASE 1

Vague Memo / Reference

Payment arrives with memo "Thank you" or "Services rendered." No invoice number. Need to match purely on amount + customer identity.

HARD CASE 2

Multi-Invoice Payment

Customer pays $14,237 which is the exact sum of INV-042 ($8,000) + INV-043 ($6,237). System must detect this combination.

HARD CASE 3

Partial Payment

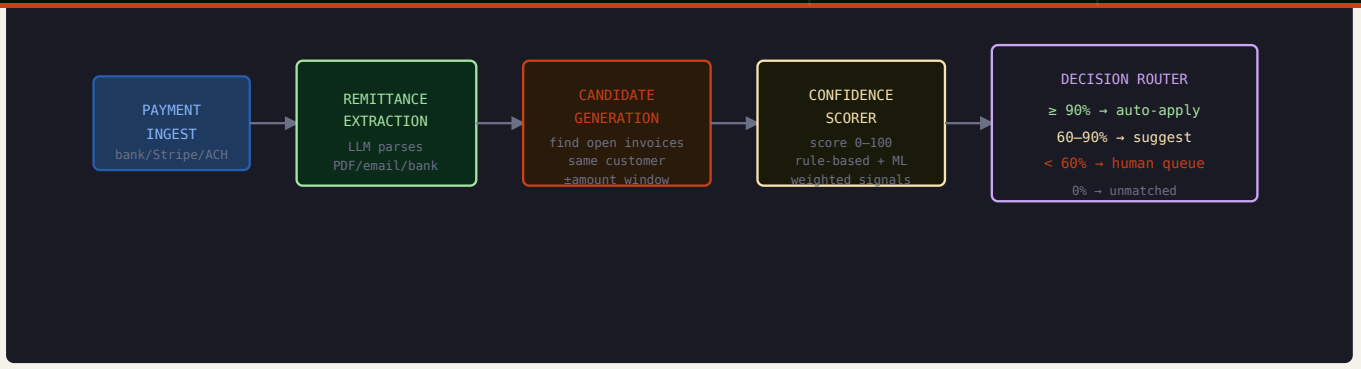
Customer pays $4,000 against a $5,000 invoice. Is this a partial payment (leave $1,000 outstanding) or a dispute (mark the invoice disputed)?

HARD CASE 4

Overpayment

Customer pays $5,100 on a $5,000 invoice. Is it a rounding error to ignore, or $100 credit to apply to the next invoice?

MATCHING PIPELINE ARCHITECTURE



CONFIDENCE SCORING SIGNALS

SIGNAL	WEIGHT	NOTES
Exact invoice number in memo	**+50**	"INV-042" found in payment memo → near certain
Exact amount match	**+30**	Payment amount = invoice amount precisely
Customer identity match	+20	Payment payer name matches customer name/entity
Amount within 2% tolerance	+10	Handles FX rounding, bank fees
Fuzzy match on customer name	+10	"Acme Corp" vs "ACME CORPORATION" — still a match
Payment date near due date	+5	Payment arrived around when the invoice was due
Multi-invoice sum match	+25	Payment = sum of 2+ open invoices (check combinations)
Prior payment history	+5	Customer has always paid invoice X on this date

SCHEMA

```
TABLE payments ( id UUID PRIMARY KEY, tenant_id UUID, amount_cents BIGINT, currency
TEXT, payer_name TEXT, memo TEXT, received_at TIMESTAMPTZ, source TEXT, -- 'stripe'
| 'ach' | 'wire' | 'check' match_status
ENUM('unmatched','auto_matched','suggested','human_review','confirmed'),
raw_payload JSONB ); TABLE payment_allocations ( id UUID PRIMARY KEY, payment_id
UUID REFERENCES payments, invoice_id UUID REFERENCES invoices, allocated_cents
BIGINT, -- supports partial + multi-invoice confidence INT, -- 0–100 score
```

MULTI-INVOICE COMBINATION PROBLEM

When a single payment covers multiple invoices, you need to find which combination of open invoices sums to the payment amount. This is the **subset sum problem** — NP-complete in theory, but tractable in practice because:

```
1. Limit search to customer's open invoices only (usually < 20)
2. Sort by amount descending, greedy first pass
3. If greedy fails, try combinations up to depth 4 (covers 99% of real cases)
4. Apply amount tolerance (±2%) to handle FX and bank fees
5. Flag anything unresolved for human review
```

🎙 QUESTIONS JOE WILL ASK YOU

- → A payment exactly matches two different invoices — what do you do?
- → How do you handle FX payments (USD invoice, EUR payment)?
- → The human reviewer confirms a wrong match — how do you undo it?
- → How do you measure matching accuracy over time?
- → At what confidence threshold do you auto-apply vs require human review?
- → How does a customer see what was matched and why?

🙋 QUESTIONS YOU SHOULD ASK JOE

- → What's Monk's current auto-match rate — what % goes to human review?
- → What's the most common failure mode in matching today?
- → Do customers use bank feeds, or do they upload remittances manually?
- → Is the matching model purely rule-based or is there an ML component?

⚡ CRITICAL KNOWLEDGE TO HAVE COLD

- ✦ Subset sum problem (and why it's tractable here)
- ✦ Fuzzy string matching (Levenshtein, trigram similarity)
- ✦ Confidence scoring / weighted signals pattern
- ✦ Human-in-the-loop queuing design

- Postgres pg_trgm for fuzzy text search
- FX / currency normalization

> 💡 **FOUNDER INSIGHT — Say This**
>
> *"The confidence threshold isn't just a technical parameter — it's a business decision about risk tolerance. A false positive (matching the wrong invoice) is worse than a false negative (sending it to human review), because it corrupts your AR records and destroys customer trust. I'd start the auto-apply threshold conservatively at 90% and tune it down as you validate accuracy with real data."*